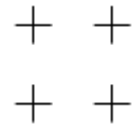
At the end of 1999, France was confronted by a huge storm which destroyed many parts of our forests. One of the most emblematic was the park of the Versailles palace, where hundreds of old trees fell. It took much effort to clean up fallen trees.

At the time, the age of the trees was reevaluated. The method used was to count the number of rings appearing in each tree's cross-section. The more rings, the older the tree is. To speed up the job, pictures of all trees were taken. Now, your work is to process the pictures and deduce the age of the trees.

The input for your program is a list of some points present in rings. The rings are no longer available, but you can reconstruct them by computing a series of convex hulls. To find a first ring, you can compute the convex hull of all points. Then, to find the next ring, you can remove those points on the hull and compute the convex hull of the remaining points, and so on. When there are no points left, the age of the tree is given by the number of hulls found.

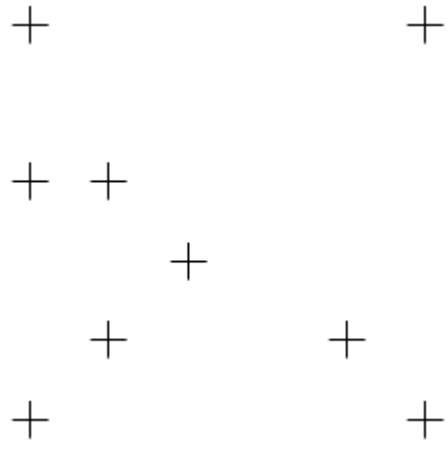
Let us consider Example 1:



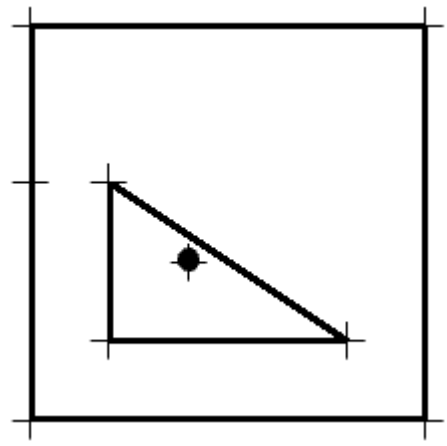
Here, there is just one ring:



Example 2 is more intricate:



The rings are:



Hence the age is 3.

Input

The input file consists of several test cases. Each of them contains lines of integers. The first line contains one integer, the number n of points of the sample ($1 \leq n \leq 3000$). The next n lines each contains two integers, the abscissa x_i and ordinate y_i of the point. All points are pairwise distinct and have coordinates in -50000..50000.

Output

For each test case of input, the output file will consist of one line containing one integer, the age of the tree.

Sample Input

```
4
0 0
1 0
1 1
0 1
9
0 0
0 3
5 0
5 5
0 5
1 1
1 3
4 1
2 2
```

Output

```
1
3
```

Charles Frédéric Gros (CFG) has decided to disprove the Riemann hypothesis numerically. For a given integer $D > 0$ of the form $4k + 3$ and free of square prime factors, this amounts to computing the cardinality $h(D)$ of the set

$$C(D) \stackrel{\text{def}}{=} \{(a, b, c) | b^2 - 4ac = -D, |b| \leq a \leq c, \text{ where } b \geq 0 \text{ if } a = c \text{ or } a = |b|.$$

(Where a, b, c are integers.)

For instance, $C(3) = \{(1, 1, 1)\}$, $C(15) = \{(1, 1, 4), (2, 1, 2)\}$. Note that $D = 75$ is not eligible, since $75 = 3 \cdot 5^2$. Non-eligible numbers in the interval $[3, 103]$ are $\{27, 63, 75, 99\}$.

CFG is interested in values of D for which $h(D)/\sqrt{D}$ is large. Your role is to write a program to help CFG finding these record numbers.

Input

You are given an input file consisting of several test cases, each of them consists of three integers on a single line:

Dmin Dmax K

where $3 \leq Dmin \leq Dmax < 2^{31}$ and are of the form $4k + 3$. Moreover, $Dmax - Dmin \leq 10^6$ and $K < 10^4$. For such values, one has $h(D) < 2^{31}$.

Output

For each test case, your program must determine the eligible values of D in the interval $[Dmin, Dmax]$ for which

$$f(D) = \left\lfloor (1000\ h(D))/\lfloor\sqrt{D}\rfloor \right\rfloor \geq K.$$

The output will consist of lines:

D h f

where D is a record number, $h = h(D)$ and $f = f(D)$.

If no answer is found, then output a line containing the word '`empty`'.

Write a blank line to separate the output of two consecutive cases.

Sample Input

```
3 103 0
27 27 10
```

Sample Output

```
3 1 1000
7 1 500
11 1 333
15 2 666
19 1 250
23 3 750
31 3 600
35 2 400
39 4 666
43 1 166
47 5 833
51 2 285
55 4 571
59 3 428
67 1 125
71 7 875
79 5 625
83 3 333
87 6 666
91 2 222
95 8 888
103 5 500

empty
```

Wireless networking is our future, provided at least some connections exist!

More precisely, N nodes of a radio network are distributed in a $L \times H$ area. Two nodes may communicate if their euclidean distance is strictly less than R, the radio range. We then say that there exists a wireless link between those two nodes. Of course, we do not consider that a node has a link with itself.

Write a program that, given a description of the positions of the nodes, outputs the number of wireless links in the network.

Input

The input file consists of several test cases, each of them as described below.

The positions (x, y) of the nodes are integers in the ranges $0 \leq x < L$ and $0 \leq y < H$. The first line of the input consists in the four integers L, H, R and N, with $0 < L \leq 5 \cdot 10^6$, $0 < H \leq 5 \cdot 10^6$, $0 < R \leq 30000$, $0 < N \leq 300000$.

Then come N lines with the coordinates x and y of each node. All node positions are different. You can assume that each node has links with at most 31 nodes.

Output

For each input case, the output consists in a single line containing the number of wireless links in the network.

Sample input

```
10 10 5 3
0 0
4 0
0 5
30 20 11 6
0 0
0 10
10 0
10 10
20 0
20 10
```

Sample Output

```
1
7
```

Last summer was so hot that it was impossible to do anything but stay in the bathroom and solve puzzles. These games were so boring that you decided to write programs to solve them. This gave you a good reason to go back to the lab and benefit from its air conditioner.

Here is the game you decided to solve automatically. Take a grid with n rows and m columns, with a letter in each cell. From the grid's cells, one can read words horizontally, vertically or diagonally, oriented forwards or backwards. You also have a list of words, and the goal is to highlight all occurrences of these words on the grid. The letters in non-highlighted cells of the grid form a word (called the *code word*) that can be read from top to bottom and left to right. The goal of the program is to find the code word automatically.

Input

The input file contains several test cases.

Each case begins with the two integers n and m on a line ($0 < n, m \leq 40$), separated by a blank. The second line contains the number k ($0 < k \leq 200$) of words in the list. Then follow the k words, one per line. Words do not have more than 22 letters. After these, follows the grid, that is to say n rows filled with m uppercase letters.

Output

For each case, write the code word on a line.

Note: The sample input data corresponds to the array:

B B A R E D
A C A L H F
R W O E I I
E R O S N N
D A R N D E
A L L Y G S

Looking for the word `ALLY`, we find it at the bottom:

B B A R E D
A C A L H F
R W O E I I
E R O S N N
D A R N D E
A L L Y G S

After highlighting it, we look now for `BARE` that appears twice:

B B A R E D
A C A L H F
R W O E I I
E R O S N N
D A R N D E
A L L Y G S

We then treat `BARED` that also appears twice:

B B A R E D
A C A L H F
R W O E I I
E R O S N N
D A R N D E
A L L Y G S

Proceeding until the end of the list, we are left with:

C A
E
S
A R

Thus, it results as sample output.

Sample Input

```
6 6
7
ALLY
BARE
BARED
FINES
HIND
LORD
WONG
BBARED
ACALHF
RWOEII
EROSNN
DARNDE
ALLYGS
```

Sample Output

```
CAESAR
```

You are back in the bathroom, now solving standard crosswords puzzles. Did you ever think about producing such puzzles? Producing crossword puzzles is surprisingly tedious, especially if one considers that solving crossword puzzles is surprisingly amusing. Fortunately, given a dictionary of words, your computer can help to produce lots of crosswords puzzles.

p	l	a	s	m	a
l	i	c	h	e	n
a	c	h	i	n	g
s	h	i	n	t	o
m	e	n	t	o	r
a	n	g	o	r	a

The puzzles we consider are special: frames are $n \times n$ empty squares and filled frames are symmetric with respect to the descending diagonal. That is, the i-th "down" word and the i-th "across" word are the same (see above for an example on a 6×6 frame).

Write a program that reads a list of words from standard input and writes how many puzzles can be produced from those words.

Input

The input file contains several test cases.

The first line of each one contains two integers n ($2 \leq n \leq 16$) and m ($1 \leq m \leq 20000$). Integer n is the frame size while integer m is the number of words in the dictionary. Each of the next m lines contains a word of n letters. Those words are made of the 26 lowercase letters (from '`a`' to '`z`') and no word may appear more than once.

Output

For each input case, the first and only line of the output should contain an integer r, the number of different puzzles that can be made with the words of the list. Notice that a given puzzle may contain a word more than once.

Note: The 8 puzzles for the first input case are:

```
aaaa   aaaa   abab   abab   baba   baba   bbbb   bbbb
aaaa   abab   baba   bbbb   aaaa   abab   baba   bbbb
aaaa   aaaa   abab   abab   baba   baba   bbbb   bbbb
aaaa   abab   baba   bbbb   aaaa   abab   baba   bbbb
```

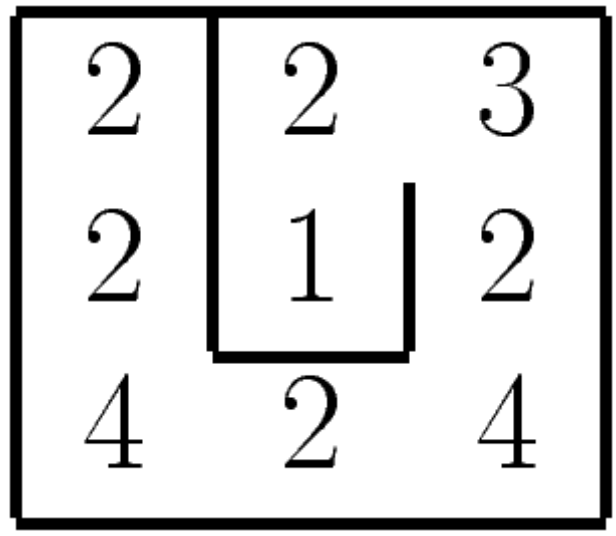
Sample Input

```
4 4
aaaa
bbbb
abab
baba
16 1
abcdefghijklmopq

8
0
```

Usually, labyrinths are to be solved by finding a path from entry to exit. This time, you are to discover the labyrinth itself...

A labyrinth is laid on a square $N \times N$ grid, and hence contains N^2 cells. Every cell holds the total number of cells that are visible both vertically and horizontally, the walls being opaque. For instance, in the picture above, the center cell (second row, second column) holds "1", since there are walls surrounding the cell in all directions except above, where the next wall is one cell away. Similarly, the leftmost-bottommost cell (third row, first column) holds "4" since, from this cell, one sees 2 cells above, 2 other cells to the right, none below and none to the left.



Given the numbers in the cells, find the labyrinth. You may assume that the square grid is surrounded by walls and that the problem has a solution.

Input

The input file contains several test cases, each of then defined as follows:

The first line consists of number N, with $1 \leq N \leq 16$. Then come N lines, each containing the N integers of a row.

Output

For each input case, output an ASCII representation of the labyrinth. This representations is made of $2N + 1$ lines of $2N + 1$ characters. For instance the labyrinth above should be given as:

```
+-+-+-+
| |   |
+ + + +
| | | |
+ +-+ +
|     |
+-+-+-+
```

Notice that walls are represented either by '-' or '|' depending on whether they are horizontal or vertical. Cells and lack of walls are rendered by ordinary spaces, while (potential) walls meet on '+'.

Write a blank line to separate the output of two consecutive cases.

Sample Input

```
2
2 2
2 2
4
5 6 3 4
3 4 2 1
5 6 5 4
1 4 2 1
```

Sample Output

```
+-+-+
|   |
+ + +
|   |
+-+-+

+-+-+-+-+
|       |
+ + +-+ +
|   | | |
+ + + +-+
|       |
+-+ + + +
|   | | |
+-+-+-+-+
```

In many places, the card game "war" is popular amongst children. As we shall see, the game result completely depends on the initial distribution of cards between two players, A and B.

We consider a standard 32 card game and we ignore suits, only ranks are important. Hence we have 4×8 cards: seven (`7`), eight (`8`), nine (`9`), ten (`T`), jack (`J`), queen (`Q`), king (`K`) and ace (`A`) — ace being the strongest card and seven the weakest, *i.e.* `7` < `8` < `9` < `T` < `J` < `Q` < `K` < `A`.

At game start, players each hold a deck of 16 cards in their hand, face down. The object of the game is to win all the cards. Both players now turn their top card face up and put it on the table. Whoever plays the higher card takes both cards and adds them (face down) to the bottom of his or her deck. It is important to notice that the higher, winning, card is added first. Then both players turn over their next card, and so on.

If the turned over cards are equal there is a war. The tied cards stay on the table and both players play the next card of their deck. Whoever has the higher of the new face-up cards wins the war and adds all four cards face-down to the bottom of his or her deck. If the new face-up cards are equal as well, the war continues: each player puts another card face-up. The war goes on like this as long as the face-up cards continue to be equal. As soon as they are different the player of the higher card wins all the cards in the war and add those cards to the bottom of his or her deck. The higher, winning, card is added first, then the lower card, then all the tied card in reverse playing order.

For instance, let us assume that players A and B have the following decks (the deck of player A is shown first and top of decks at left):

```
AKKQQJJTT998877A
AK7QQJJTT99887KA
```

A war is started because both A and B play aces, then the war goes on because of kings, and finally player A's king wins over player B's seven. Hence, player A collects all cards on the table. Now, the decks are as follows:

```
QQJJTT998877AK7KKAA
QQJJTT99887KA
```

As regards the outcome of the whole game there are four situations:

1. Player A wins the game when player B's deck becomes empty while player A still owns cards.
2. Player B wins the game in the symmetric situation.
3. There is a draw, when both players exhaust their cards simultaneously.
4. The game lasts forever.

Write a program that computes the outcome of the game from the initial decks of players A and B.

Input

The input file contains several test cases, each of them is made of two lines: the first line is player A's deck, while the second line is player B's deck.

Output

For each test case, your program must output one line, which should be '`A wins`', '`B wins`', '`draw game`' or '`play forever`', depending on the game's outcome.

Sample Input

```
AKKQQJJTT998877A
AK7QQJJTT99887KA
AAKKQQJJTT998877
AAKKQQJJTT998877
87TT8799QJAAQJKK
T8T79897AQAJKQKJ
```

Sample Output

```
A wins
draw game
play forever
```

As you must have experienced, instead of landing immediately, an aircraft sometimes waits in a holding loop close to the runway. This holding mechanism is required by air traffic controllers to space apart aircraft as much as possible on the runway (while keeping delays low). It is formally defined as a "holding pattern" and is a predetermined maneuver designed to keep an aircraft within a specified airspace (see Figure 1 for an example).

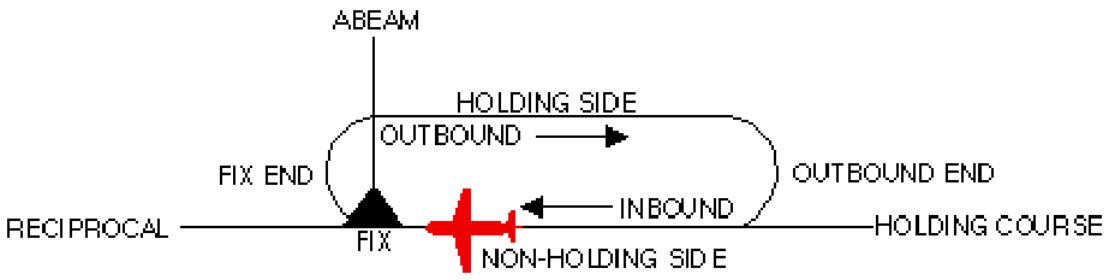


Figure 1: A simple Holding Pattern as described in a pilot text book.

Jim Tarjan, an air-traffic controller, has asked his brother Robert to help him to improve the behavior of the airport.

The TRACON area

The Terminal Radar Approach CONtrol (TRACON) controls aircraft approaching and departing when they are between 5 and 50 miles of the airport. In this final scheduling process, air traffic controllers make some aircraft wait before landing. Unfortunately this "waiting" process is complex as aircraft follow predetermined routes and their speed cannot be changed. To reach some degree of flexibility in the process, the basic delaying procedure is to make aircraft follow a holding pattern that has been designed for the TRACON area. Such patterns generate a constant prescribed delay for an aircraft (see Figure 1 for an example). Several holding patterns may exist in the same TRACON.

In the following, we assume that there is a *single runway* and that when an aircraft enters the TRACON area, it is assigned an *early landing time*, a *late landing time* and a possible holding pattern. The early landing time corresponds to the situation where the aircraft does not wait and lands as soon as possible. The late landing time corresponds to the situation where the aircraft waits in the prescribed holding pattern and then lands at that time. We assume that an aircraft enters at most one holding pattern. Hence, the early and late landing times are the only two possible times for the landing.

The *security gap* is the minimal elapsed time between consecutive landings. The objective is to *maximize the security gap*. Robert believes that you can help.

Example

Assume there are 10 aircraft in the TRACON area. Table 1 provides the corresponding early and late landing times (columns "Early" and "Late").

Aircraft	Early	Late	Solution
A_1	44	156	Early
A_2	153	182	Early
A_3	48	109	Late
A_4	160	201	Late
A_5	55	186	Late
A_6	54	207	Early
A_7	55	165	Late
A_8	17	58	Early
A_9	132	160	Early
A_{10}	87	197	Early

Table 1: A 10 aircraft instance of the problem.

The maximal security gap is 10 and the corresponding solution is reported in Table 1 (column "Solution"). In this solution, the aircraft land in the following order: A_8, A_1, A_6, A_{10}, A_3, A_9, A_2, A_7, A_5, A_4. The security gap is realized by aircraft A_1 and A_6.

Input

The input file, that contains all the relevant data, contains several test cases

Each test case is described in the following way. The first line contains the number n of aircraft ($2 \leq n \leq 2000$). This line is followed by n lines. Each of these lines contains two integers, which represent the early landing time and the late landing time of an aircraft. Note that all times t are such that $0 \leq t \leq 10^7$.

Output

For each input case, your program has to write a line that conttains the maximal security gap between consecutive landings.

Note: The input file corresponds to Table 1.

Robert's Hints

Optimization vs. Decision Robert advises you to work on the decision variant of the problem. It can then be stated as follows: Given an integer p, and an instance of the optimization problem, the question is to decide if there is a solution with security gap p or not. Note that, if you know how to solve the decision variant of an optimization problem, you can build a binary search algorithm to find the optimal solution.

On decision Robert believes that the decision variant of the problem can be *modeled as a very particular boolean satisfaction problem*. Robert suggests to associate a boolean variable per aircraft stating whether the aircraft is early (variable takes value "true") or late (value "false"). It should then be easy to see that for some aircraft to land at some time has consequences for the landing times of other aircraft. For instance in Table 1 and with a delay of 10, if aircraft A_1 lands early, then aircraft A_3 has to land late. And of course, if aircraft A_3 lands early, then aircraft A_1 has to land late. That is, aircraft A_1 and A_3 cannot both land early and formula $(A_1 \Rightarrow \neg A_3) \wedge (A_3 \Rightarrow \neg A_1)$ must hold.

And now comes Robert's big insight: our problem has a solution, if and only if we have no contradiction. A contradiction being something like $A_i \Leftrightarrow \neg A_i$.

Sample Input

```
10
44 156
153 182
48 109
160 201
55 186
54 207
55 165
17 58
132 160
87 197
```

Sample Output

```
10
```

The city of X (for reasons that will become clear, the name of this city is kept secret) is one of the safest cities in the country. However, the mayor of X has decided to eliminate crime completely.

The Perfect Police Patrol System (PPPS)

The Perfect Police Patrol System is the mayor's response to crime. According to the mayor's analysis, the city consists of intersections and streets, where every street connect two intersections. Crime must be eliminated from the streets.

The mayor's idea is to install PPPS stations at some intersections. The officers in a station can patrol all the streets that enter the intersection where the station is, and only those. To achieve maximal security every street should be patrolled. That is, *there must be a PPPS station at one of the two ends of every street.* However, in the past, there were unfortunate incidents where two police patrols fought with each other in dark streets, because they could not recognize that they were both from the police. To avoid such untimely incidents, *there cannot be PPPS stations at both ends of a street.* Finally, as PPPS stations are very expensive and as the mayor is not willing to increase taxes, *one should build as few PPPS stations as possible.*

Write a program that determines whether it is possible to design such a Perfect Police Patrol System. If so, the program should output the minimal number of stations that have to be built.

Input

The input contains several test cases, each of them consists of lines of integers. The first line contains two integers n and m ($1 \leq n \leq 1000$, $1 \leq m \leq 10000$) where integer n is the number of intersections and integer m is the number of streets. The next m lines describe the m streets. Each of theses lines contains two integers that identify the two intersections at the two ends of the streets (intersections are numbered $1, 2, \ldots, n$). Because of the various tunnels, bridges, temporarily closed streets etc. you cannot assume anything about the topology of the city.

Output

For each test case, the output contains one single line. If the PPPS can be designed, then the program should output one integer, the minimal number of PPPS stations required. Otherwise, if no PPPS can be designed, the program should output the line '`Impossible`' (without quotes).

Sample Input

```
7 4
1 2
1 3
4 5
6 5
5 6
1 2
1 3
2 4
3 4
3 5
4 5
```

Sample Output

```
2
Impossible
```